

学习内容

CONTENTS

1

机器学习概述

2

线性回归与逻辑回归

3

支持向量机

4

聚类与降维

5

决策树

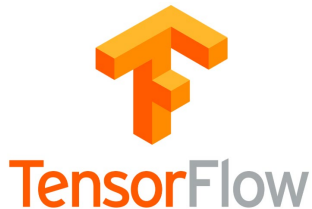
6

强化学习



Python在人工智能中的“武器库”（核心工具链）：

- 1) Numpy：科学计算基石
- 2) Pandas：数据处理瑞士军刀
- 3) Matplotlib：数据可视化
- 4) Scikit-learn：机器学习入门必备
- 5) PyTorch：深度学习框架



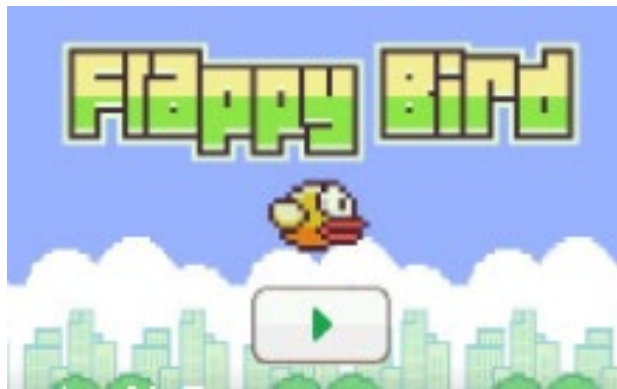
Python语言是人工智能领域中具有天然优势的编程语言

什么是强化学习？

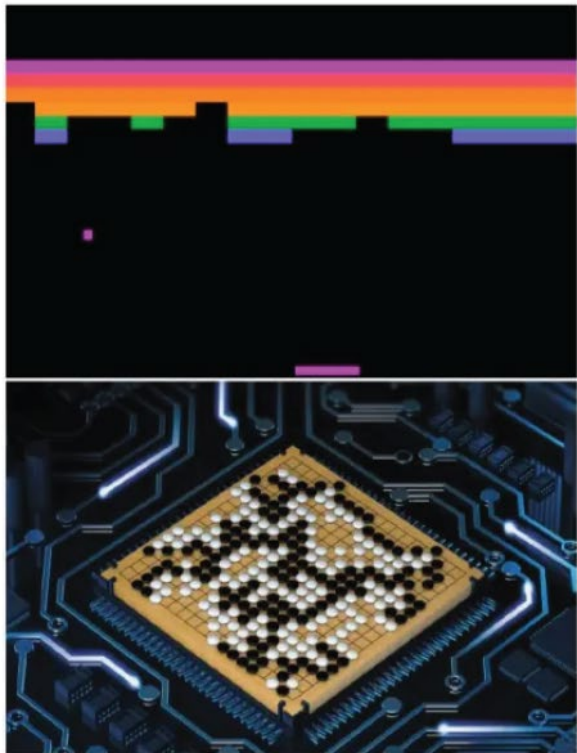
- » **强化学习 (Reinforce Learning)**: 智能体 (agent) 在不断与其所处环境交互的过程中进行学习的一种方法。
- » **强化学习目的: 使智能体学习制定正确的决策顺序** (Learn to make good sequences of decisions) 。

如: 让计算机学着去玩Flappy Bird

我们不需要设置具体的策略, 比如先飞到上面, 再飞到下面, 我们只是需要给算法定一个“小目标”! 比如当计算机玩得好的时候, 我们就给它一定的奖励, 它玩得不好的时候, 就给它一定的惩罚, 在这个算法框架下, 它就可以越来越好, 超过人类玩家的水平。

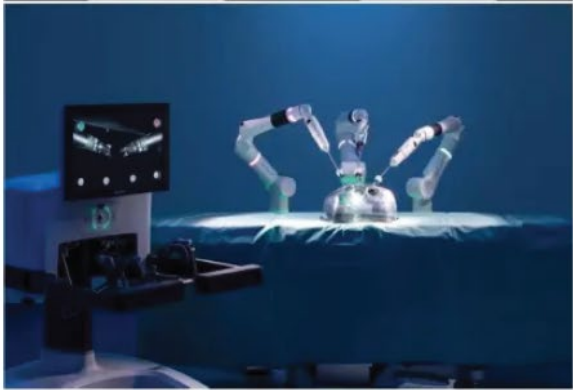




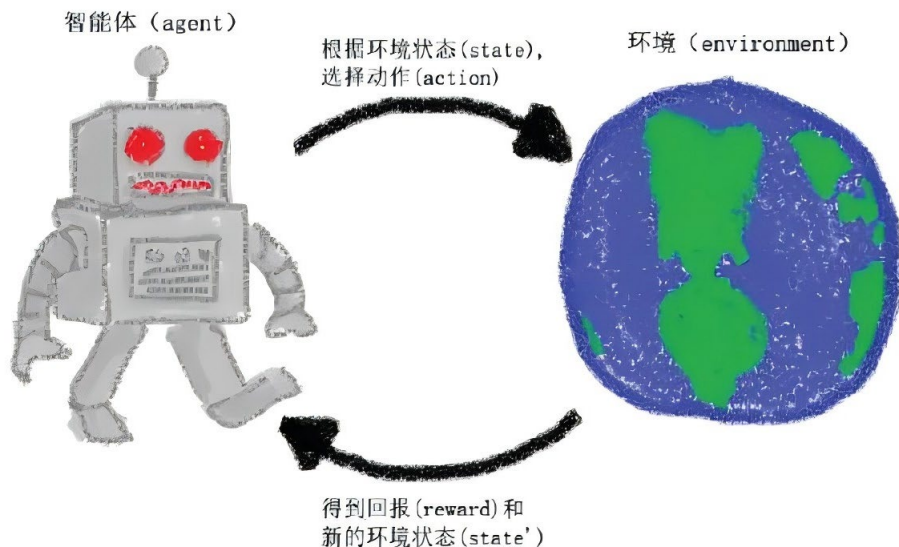


里程碑：突破 2600 个 Atari 游戏环境、AlphaGo 与 AlphaGoZero、
Libratus 与 DeepStack、AlphaStar

应用：人类行为模拟、机器人医疗、机器狗、自动驾驶

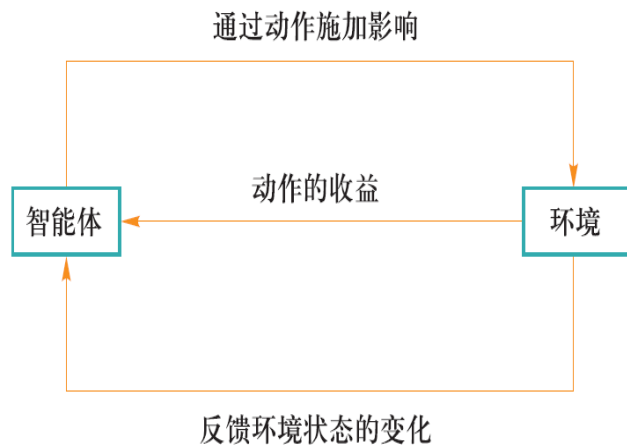


强化学习：在环境交互中进行学习

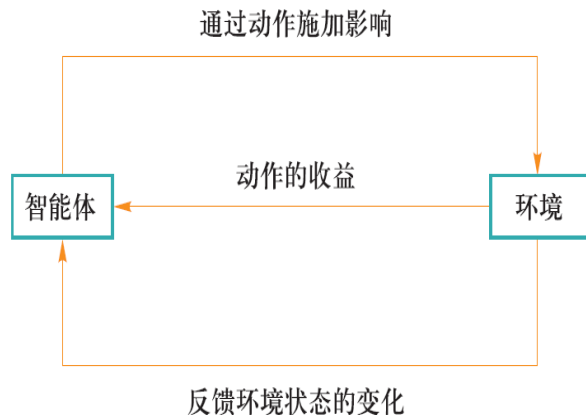


» 智能体通过“尝试与试错”和“探索与利用”等机制在所处状态采取行动，不断与环境交互，直至进入终止状态，根据在终止状态所获得的奖惩来改进行动策略，序贯完成决策任务。

- **智能体 (agent)**：智能体是强化学习算法的主体，它能够根据经验做出主观判断并执行动作，是整个智能系统的核心。
- **环境 (environment)**：智能体以外的一切统称为环境，环境在与智能体的交互中，能被智能体所采取的动作影响，同时环境也能向智能体反馈状态和奖励。
- **状态 (state)**：状态可以理解为智能体对环境的一种理解和编码，通常包含了对智能体所采取决策产生影响的信息。

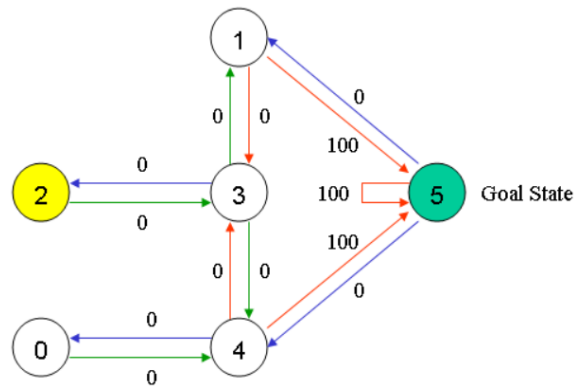


- **动作 (action)** : 动作是智能体对环境产生影响的方式
- **策略 (policy)** : 策略是智能体在所处状态下去执行某个动作的依据, 即给定一个状态, 智能体可根据一个策略来选择应该采取的动作。
- **奖励 (reward)** : 奖励是智能体序贯式采取一系列动作后从环境获得的收益。

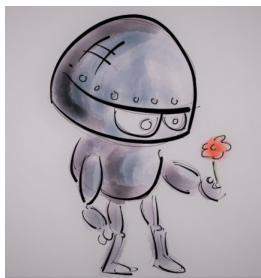




某建筑房间分布平面图



奖励值



奖励矩阵

奖励矩阵 $R = \begin{bmatrix} 1 & 1 & 1 & 0 & 1 & 100 \\ -1 & -1 & -1 & 0 & -1 & -1 \\ -1 & 0 & 0 & -1 & 0 & -1 \\ 0 & -1 & -1 & 0 & -1 & 100 \\ -1 & 0 & -1 & -1 & 0 & 100 \end{bmatrix}$

- [-1]: 状态之间是不互通;
- [0]: 状态之间互通, 奖励为0;
- [100]: 状态之间互通, 奖励为100。

算法原理

- Q-Learning : 一种基于**值**迭代的**无模型**强化学习算法，由Watkins于1989年提出。它不需要事先了解环境模型，而是通过试错来学习最优策略。
- 核心目标：通过与环境的交互，学习一个最优策略（Policy），使得智能体（Agent）在长期交互中获得最大累积奖励。
- 核心思想：通过Q表来存储状态-动作对应的Q值，**Q值**代表在某个状态下采取某个动作所能获得的长期回报。

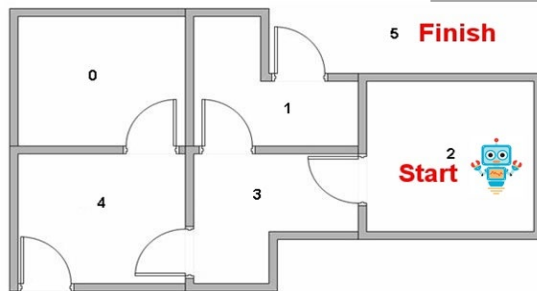


- 学习和奖励矩阵 R 同阶的知识矩阵 Q
- 假设状态数目已知

Q 为状态-行动值函数 ($Q(s,a)$ 函数)，用于评价在特定状态 s 下采取某个动作 a 的优劣

	行动0	行动1	行动2	行动3	行动4	行动5
状态0	0	0	0	0	0	0
状态1	0	0	0	0	0	0
状态2	0	0	0	0	0	0
状态3	0	0	0	0	0	0
状态4	0	0	0	0	0	0
状态5	0	0	0	0	0	0

初始化 Q 为全零矩阵



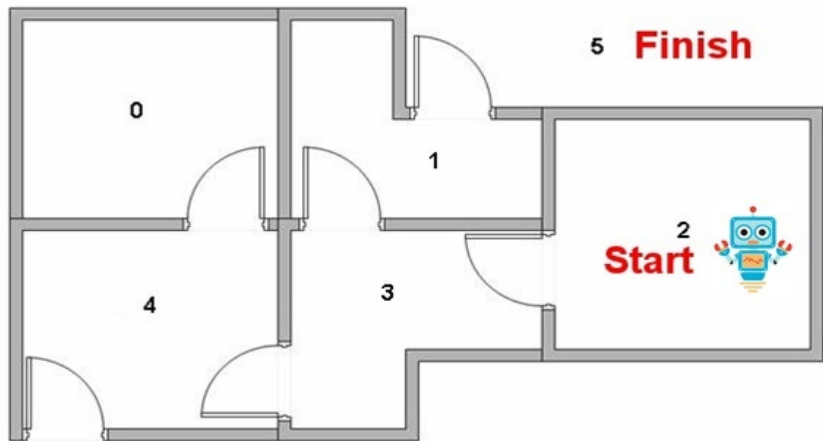
$$Q = \begin{matrix} \begin{matrix} 0 \\ 1 \\ 2 \\ 3 \\ 4 \\ 5 \end{matrix} & \begin{bmatrix} 0 & 0 & 0 & 0 & 80 & 0 \\ 0 & 0 & 0 & 64 & 0 & 100 \\ 0 & 0 & 0 & 64 & 0 & 0 \\ 0 & 80 & 51 & 0 & 80 & 0 \\ 64 & 0 & 0 & 64 & 0 & 100 \\ 0 & 80 & 0 & 0 & 80 & 100 \end{bmatrix} \end{matrix}$$

最终学习得到Q-table

$$R = \begin{matrix} \begin{matrix} \text{State} \\ 0 \\ 1 \\ 2 \\ 3 \\ 4 \\ 5 \end{matrix} & \begin{matrix} \text{Action} \\ 0 & 1 & 2 & 3 & 4 & 5 \end{matrix} & \begin{bmatrix} -1 & -1 & -1 & -1 & 0 & -1 \\ -1 & -1 & -1 & 0 & -1 & 100 \\ -1 & -1 & -1 & 0 & -1 & -1 \\ -1 & 0 & 0 & -1 & 0 & -1 \\ 0 & -1 & -1 & 0 & -1 & 100 \\ -1 & 0 & -1 & -1 & 0 & 100 \end{bmatrix} \end{matrix}$$

Agent根据当前所在房间 (状态 s) 在 Q 矩阵中查找到对应的行, 选择值最大的动作 a 作为当前动作, 就可以顺利到达目标房间。

- **状态 (State) s** : 智能体所处的环境描述 (如所在房间)。
- **动作 (Action) a** : 智能体可执行的操作 (如去向某个房间)。
- **奖励 (Reward) R** : 环境对智能体动作的**即时反馈** (如到达目标房间+100, 其它房间+0)。
- **Q值 (Q-Value)**: 表示在状态 s 下选择动作 a 的**预期长期回报**。



1. 给定参数 α 、 γ 和奖励矩阵 R
2. 初始化 Q 表，即令 $Q = 0$
3. 迭代执行以下步骤，直到 Q 收敛：

a) 随机选择一个初始状态 s

b) 如果 s 不是目标状态，则执行： $\hat{a} = \arg \max_{\tilde{a}} Q(s, \tilde{a})$

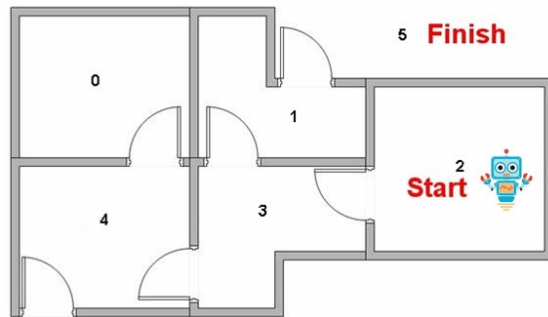
c) 记执行 $\hat{a}$ 后的新状态为 $\hat{s}$

d) 更新 Q 值：

$$Q(s, \hat{a}) = Q(s, \hat{a}) + \alpha(R(s, \hat{a}) + \gamma \cdot \max_{a'} Q(\hat{s}, a') - Q(s, \hat{a}))$$

e) 更新状态： $s = \hat{s}$

f) 如果 s 为目标状态
则结束，否则返回b)



更新步长

立即奖励/眼前利益

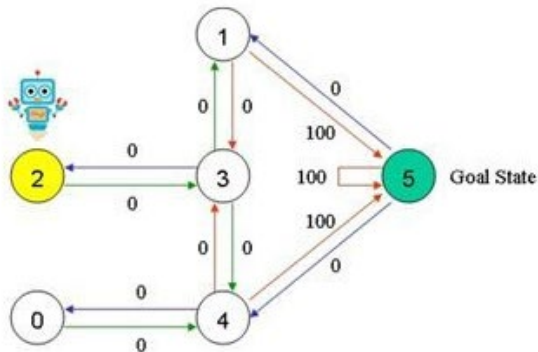
奖赏折扣在 $[0,1]$ 之间，表示侧重当前奖励还是未来奖励。

未来奖励：根据以往经验对新状态给出的最大评价

【参数设置: $\alpha=1$, $\gamma=0.8$ 】

$$Q(s, \hat{a}) = Q(s, \hat{a}) + \alpha (R(s, \hat{a}) + \gamma \cdot \max_{a'} Q(\hat{s}, a') - Q(s, \hat{a}))$$

First: 随机选择初始状态1



State	Action					
	0	1	2	3	4	5
0	-1	-1	-1	-1	0	-1
1	-1	-1	-1	0	-1	100
2	-1	-1	-1	0	-1	-1
3	-1	0	0	-1	0	-1
4	0	-1	-1	0	-1	100
5	-1	0	-1	-1	0	100

$$Q = \begin{matrix} & \begin{matrix} 0 & 1 & 2 & 3 & 4 & 5 \end{matrix} \\ \begin{matrix} 0 \\ 1 \\ 2 \\ 3 \\ 4 \\ 5 \end{matrix} & \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix} \end{matrix}$$

Q-table

$$Q = \begin{matrix} & \begin{matrix} 0 & 1 & 2 & 3 & 4 & 5 \end{matrix} \\ \begin{matrix} 0 \\ 1 \\ 2 \\ 3 \\ 4 \\ 5 \end{matrix} & \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 100 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix} \end{matrix}$$

update Q-table

First round

第一步: 在R-table的第二行 ($s=$ 状态1) 下一跳能到达状态3、5;

第二步: 根据 $\hat{a} = \arg \max_{\tilde{a}} Q(s, \tilde{a})$ 随机选择下一跳 (假设为状态5);

第三步: 记新状态 $\hat{S}=5$, 更新Q-table, 因为5是目标状态, 所以我们完成了一轮。

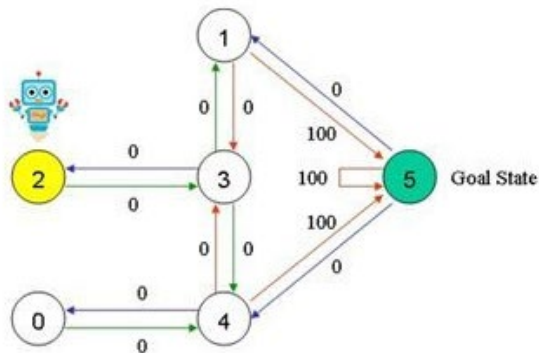
$$Q(s, \hat{a}) = R(s, \hat{a}) + \gamma \cdot \max_{a'} Q(\hat{s}, a')$$

$$Q(1,5) = R(1,5) + 0.8 * \max [Q(5,1), \dots, Q(5,5)] = 100 + 0.8 * 0 = 100$$

【参数设置: $\alpha=1$, $\gamma=0.8$ 】

$$Q(s, \hat{a}) = Q(s, \hat{a}) + \alpha(R(s, \hat{a}) + \gamma \cdot \max_{a'} Q(\hat{s}, a') - Q(s, \hat{a}))$$

Second: 随机选择初始状态3



State	Action					
	0	1	2	3	4	5
0	-1	-1	-1	-1	0	-1
1	-1	-1	-1	0	-1	100
2	-1	-1	-1	0	-1	-1
3	-1	0	0	-1	0	-1
4	0	-1	-1	0	-1	100
5	-1	0	-1	-1	0	100

	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	0	0	0	0	100
2	0	0	0	0	0	0
3	0	0	0	0	0	0
4	0	0	0	0	0	0
5	0	0	0	0	0	0

update Q-table

First

	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	0	0	0	0	100
2	0	0	0	0	0	0
3	0	80	0	0	0	0
4	0	0	0	0	0	0
5	0	0	0	0	0	0

Second round

第一步: 在R-table的第四行 ($s=$ 状态3) 下一跳能到达状态1、2、4;

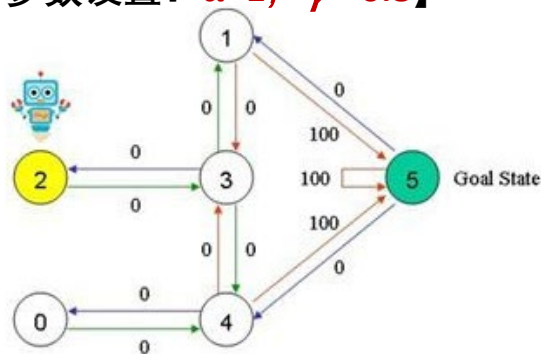
第二步: 根据 $\hat{a} = \arg \max_{\tilde{a}} Q(s, \tilde{a})$ 随机选择下一跳 (假设为状态1);

第三步: 记新状态 $\hat{S}=1$, 更新Q-table, 因为1不是目标状态, 所以返回第二步。

$$Q(s, \hat{a}) = R(s, \hat{a}) + \gamma \cdot \max_{a'} Q(\hat{s}, a')$$

$$Q(3,1) = R(3,1) + 0.8 * \max [Q(1,3), \dots, Q(1,5)] = 0 + 0.8 * \max (0,100) = 80$$

【参数设置: $\alpha=1$, $\gamma=0.8$ 】



$$R = \begin{matrix} & \text{Action} \\ \text{State} & 0 & 1 & 2 & 3 & 4 & 5 \end{matrix}$$

$$\begin{matrix} 0 \\ 1 \\ 2 \\ 3 \\ 4 \\ 5 \end{matrix} \begin{bmatrix} -1 & -1 & -1 & -1 & 0 & -1 \\ -1 & -1 & -1 & 0 & -1 & 100 \\ -1 & -1 & -1 & 0 & -1 & -1 \\ -1 & 0 & 0 & -1 & 0 & -1 \\ 0 & -1 & -1 & 0 & -1 & 100 \\ -1 & 0 & -1 & -1 & 0 & 100 \end{bmatrix}$$

First

$$Q = \begin{matrix} & 0 & 1 & 2 & 3 & 4 & 5 \end{matrix}$$

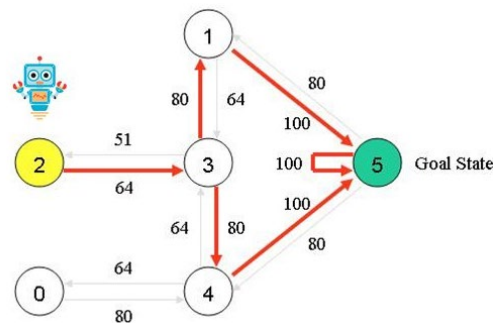
$$\begin{matrix} 0 \\ 1 \\ 2 \\ 3 \\ 4 \\ 5 \end{matrix} \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 100 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 80 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

First: 随机选择初始状态1

$$Q = \begin{matrix} & 0 & 1 & 2 & 3 & 4 & 5 \end{matrix}$$

$$\begin{matrix} 0 \\ 1 \\ 2 \\ 3 \\ 4 \\ 5 \end{matrix} \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 100 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 80 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

Second: 随机选择初始状态3



$$Q = \begin{matrix} & 0 & 1 & 2 & 3 & 4 & 5 \end{matrix}$$

$$\begin{matrix} 0 \\ 1 \\ 2 \\ 3 \\ 4 \\ 5 \end{matrix} \begin{bmatrix} 0 & 0 & 0 & 0 & 80 & 0 \\ 0 & 0 & 0 & 0 & 0 & 100 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 80 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 100 \\ 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

Four: 随机选择初始状态0

$$Q = \begin{matrix} & 0 & 1 & 2 & 3 & 4 & 5 \end{matrix}$$

$$\begin{matrix} 0 \\ 1 \\ 2 \\ 3 \\ 4 \\ 5 \end{matrix} \begin{bmatrix} 0 & 0 & 0 & 0 & 80 & 0 \\ 0 & 0 & 0 & 64 & 0 & 100 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 80 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 100 \\ 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

Fifth: 随机选择初始状态1

$$Q = \begin{matrix} & 0 & 1 & 2 & 3 & 4 & 5 \end{matrix}$$

$$\begin{matrix} 0 \\ 1 \\ 2 \\ 3 \\ 4 \\ 5 \end{matrix} \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 100 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 80 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 100 \\ 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

Third: 随机选择初始状态4

$$Q = \begin{matrix} & 0 & 1 & 2 & 3 & 4 & 5 \end{matrix}$$

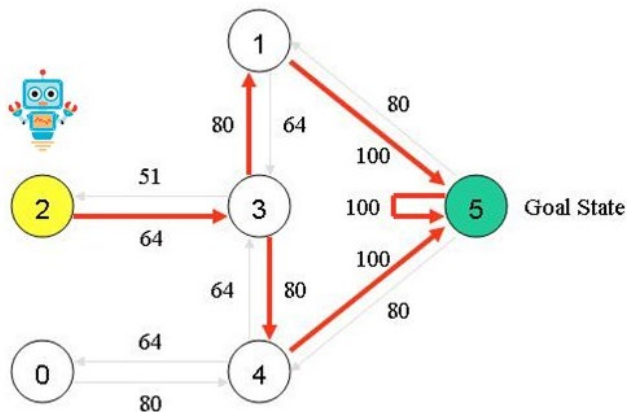
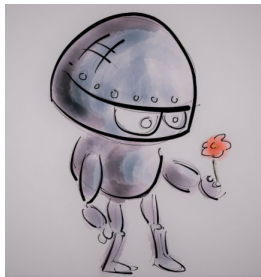
$$\begin{matrix} 0 \\ 1 \\ 2 \\ 3 \\ 4 \\ 5 \end{matrix} \begin{bmatrix} 0 & 0 & 0 & 0 & 80 & 0 \\ 0 & 0 & 0 & 64 & 0 & 100 \\ 0 & 0 & 0 & 64 & 0 & 0 \\ 0 & 80 & 51 & 0 & 80 & 0 \\ 64 & 0 & 0 & 64 & 0 & 100 \\ 0 & 80 & 0 & 0 & 80 & 100 \end{bmatrix}$$

经过循环迭代, 最终得到Q-table

1. 令当前状态 $s = s_0$
2. 选择动作 $\hat{a} = \arg \max_{\tilde{a}} Q(s, \tilde{a})$
3. 更新状态, 令 $s = \hat{s}$
4. 重复执行步骤2、3直到s成为目标状态

$$Q = \begin{matrix} & \begin{matrix} 0 & 1 & 2 & 3 & 4 & 5 \end{matrix} \\ \begin{matrix} 0 \\ 1 \\ 2 \\ 3 \\ 4 \\ 5 \end{matrix} & \begin{bmatrix} 0 & 0 & 0 & 0 & 80 & 0 \\ 0 & 0 & 0 & 64 & 0 & 100 \\ 0 & 0 & 0 & 64 & 0 & 0 \\ 0 & 80 & 51 & 0 & 80 & 0 \\ 64 & 0 & 0 & 64 & 0 & 100 \\ 0 & 80 & 0 & 0 & 80 & 100 \end{bmatrix} \end{matrix}$$

循环迭代最终得到的
Q-table



多选题：从初始状态2开始，Agent使用矩阵Q作为指导，可能的序列是（ ）。

☒ A 2 - 3 - 1 - 5

☐ B 2 - 3 - 2 - 5

☒ C 2 - 3 - 4 - 5

☐ D 2 - 3 - 5

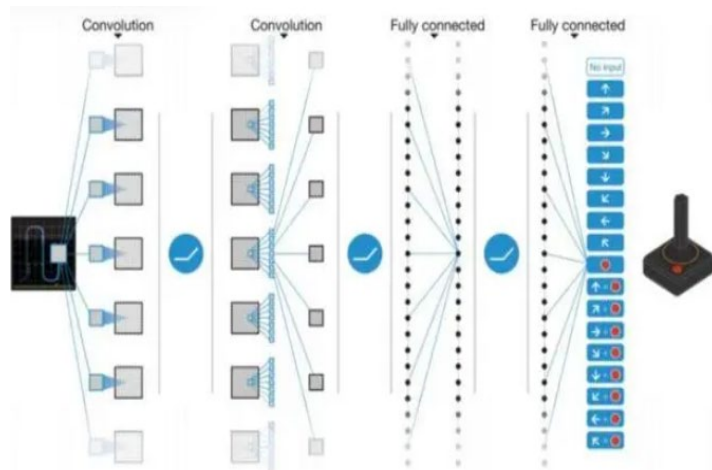
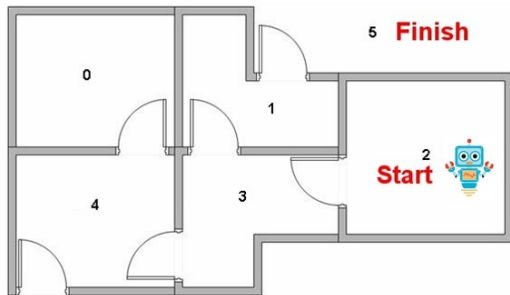
$$Q = \begin{matrix} & \begin{matrix} 0 & 1 & 2 & 3 & 4 & 5 \end{matrix} \\ \begin{matrix} 0 \\ 1 \\ 2 \\ 3 \\ 4 \\ 5 \end{matrix} & \begin{bmatrix} 0 & 0 & 0 & 0 & 80 & 0 \\ 0 & 0 & 0 & 64 & 0 & 100 \\ 0 & 0 & 0 & 64 & 0 & 0 \\ 0 & 80 & 51 & 0 & 80 & 0 \\ 64 & 0 & 0 & 64 & 0 & 100 \\ 0 & 80 & 0 & 0 & 80 & 100 \end{bmatrix} \end{matrix}$$

提交

◆ Q-learning算法是一种**无模型** (model-free) 强化学习方法，无需提前获取完备的模型，通过不断地迭代更新Q值，智能体最终可以学习到在不同状态下采取不同行动的最优策略，从而实现自主决策。

◆ Q-learning算法的主要缺点是**缺乏通用性**。如果将 Q-learning 理解为在二维数组（动作空间×状态空间）中更新数字，那么它实际上类似于动态规划。Q-learning 智能体没有能力对未见过的状态进行估值。

DQN 利用神经网络来估计 Q 值函数。



用深度 Q 网络 (DQN) 玩雅达利 (Atari) 游戏 (2013, DeepMind)



DQN in Atari

End-to-end learning of values $Q(s, a)$ from pixels

Input state s is stack of raw pixels from last 4 frames

Output is $Q(s, a)$ for 18 joystick/button positions

Reward is the change in the score for that step

Mnih, Volodymyr, et al. "Human-level control through deep reinforcement learning." *Nature* 518.7540 (2015): 529-533.

- **游戏智能体训练**：例如，在经典的Atari游戏中，Q-Learning算法可以被用来训练游戏智能体，在不断地尝试中学会如何最优化地操作游戏控制器。
- **机器人路径规划**：通过学习机器人在不同状态下采取不同行动的最优策略，可以实现机器人在复杂环境中快速、准确地找到最优路径。
- **自然语言处理**：例如，可以将自然语言问题转化为状态，将回答问题的语句转化为行动，使用Q-Learning算法来学习最优的回答策略。
- **股票交易决策**：例如，可以将不同股票价格和市场指数作为状态，将不同的交易行为（例如买进或卖出）作为行动，使用Q-Learning算法来学习最优的交易策略。





根据交互策略 (Behavior Policy) 与目标策略 (Target Policy) 是否相同区分:

- On-Policy: 用于训练当前策略 π 的样本来源于当前策略 π
- Off-Policy: 用于训练当前策略 π 的样本来源于不同的策略 μ

算法	核心特点	适用场景
Q-Learning	无模型、离线策略、Q表更新	离散状态/动作空间 (如迷宫、游戏)
SARSA	在线策略 (基于当前策略更新)	需要安全探索的场景 (如机器人控制)
Deep Q-Network	结合神经网络处理高维状态	复杂环境 (如视频游戏、自动驾驶)
Policy Gradient	直接优化策略，支持连续动作	机器人控制、自然语言生成



	有监督学习	无监督学习	强化学习
学习依据	基于监督信息	基于对数据结构的假设	基于评价 (evaluative)
数据来源	一次性给定	一次性给定	在交互中产生 (interactive)
决策过程	单步 (one-shot)	无	序列 (sequential)
学习目标	样本到语义标签的映射	同一类数据的分布模式	选择能够获取最大收益的状态到动作的映射





- 1. 请叙述强化学习的基本思想及适用场景。
- 2. 请叙述强化学习与[有监督学习](#)和[无监督学习](#)的不同之处。
- 3. Q 学习算法是一种无模型的强化学习算法，请说明 Q 函数是如何更新的？

准备机器学习实验

Python在人工智能中的“武器库”（核心工具链）：

- 1) **Numpy**：科学计算基石
- 2) **Pandas**：数据处理瑞士军刀
- 3) **Matplotlib**：数据可视化
- 4) **Scikit-learn**：机器学习入门必备
- 5) **PyTorch**：深度学习框架